

MODULE 03 SUPPLEMENT

PySpark and Spark SQL

Side-by-side reference for ELT and ingestion

Contents

SECTION 1	Reading CSV files	3
SECTION 2	Reading JSON files	5
SECTION 3	Reading Parquet files	6
SECTION 4	Exploring data	6
SECTION 5	Transforming columns	7
SECTION 6	Removing duplicates and sorting	9
SECTION 7	Conditional logic	9
SECTION 8	Filtering rows	10
SECTION 9	Aggregation	13
SECTION 10	Temporary views and SQL	15
SECTION 11	Writing to Delta	16
SECTION 12	Quick reference	17

This reference shows each operation from Module 03 in both of the course's APIs: the PySpark DataFrame API and Spark SQL. Both are compiled by the Catalyst optimizer into the same execution plan, so the choice between them is a question of readability and team preference, not of performance.

The examples use the course datasets: `customers` (10,000 rows), `orders` (100,000 rows) and `products` (2,000 rows). Volume paths are shortened to `/Volumes/.../`.

SECTION 1

Reading CSV files

Inferring the schema

PYSPARK

```
# inferSchema makes Spark read the file twice
customers_df = (
    spark.read
    .format("csv")
    .option("header", "true")
    .option("inferSchema", "true")
    .load("/Volumes/.../customers.csv")
)
```

SPARK SQL

```
-- read_files() infers the schema
SELECT *
FROM read_files(
    '/Volumes/.../customers.csv',
    format => 'csv',
    header => true
);
```

Defining the schema explicitly

PYSPARK

```
from pyspark.sql.types import StructType, StructField, StringType, DateType

schema = StructType([
    StructField("customer_id", StringType(), False),
    StructField("first_name", StringType(), True),
    StructField("email", StringType(), True),
    StructField("registration_date", DateType(), True),
])

customers_df = (
    spark.read
    .format("csv")
    .option("header", "true")
    .schema(schema)
    .load("/Volumes/.../customers.csv")
)
```

SPARK SQL

```
-- The schema is passed as a SQL string
SELECT *
FROM read_files(
    '/Volumes/.../customers.csv',
    format => 'csv',
    header => true,
    schema => 'customer_id STRING NOT NULL,
              first_name STRING,
              email STRING,
              registration_date DATE'
);
```

Creating a Bronze table

PYSPARK

```
(customers_df.write
    .format("delta")
    .mode("overwrite")
    .saveAsTable("bronze.customers"))
```

SPARK SQL

```
CREATE OR REPLACE TABLE bronze.customers AS
SELECT
    *,
    _metadata.file_path AS source_file
FROM read_files(
    '/Volumes/.../customers.csv',
    format => 'csv',
    header => true
);
```

SECTION 2

Reading JSON files

Inferring the schema

PYSPARK

```
orders_df = (  
    spark.read  
    .format("json")  
    .load("/Volumes/.../orders.json")  
)
```

SPARK SQL

```
SELECT *  
FROM read_files(  
    '/Volumes/.../orders.json',  
    format => 'json'  
);
```

Reading a subset of columns

PYSPARK

```
from pyspark.sql.types import DoubleType  
  
orders_schema = StructType([  
    StructField("order_id", StringType(), True),  
    StructField("customer_id", StringType(), False),  
    StructField("total_amount", DoubleType(), False),  
    StructField("payment_method", StringType(), False),  
)  
  
orders_df = (  
    spark.read  
    .format("json")  
    .schema(orders_schema)  
    .load("/Volumes/.../orders.json")  
)
```

SPARK SQL

```
SELECT  
    order_id,  
    customer_id,  
    total_amount,  
    payment_method  
FROM read_files(  
    '/Volumes/.../orders.json',  
    format => 'json'  
);
```

SECTION 3

Reading Parquet files

Parquet files store their own schema, so neither `inferSchema` nor a `StructType` is needed. Parquet is also the file format underneath Delta Lake, and its columnar layout lets Spark skip data it does not need.

PYSPARK

```
products_df = (  
    spark.read  
    .format("parquet")  
    .load("/Volumes/.../products.parquet")  
)
```

SPARK SQL

```
SELECT *  
FROM read_files(  
    '/Volumes/.../products.parquet',  
    format => 'parquet'  
);
```

SECTION 4

Exploring data

Operation	PySpark	Spark SQL
Preview rows	<code>df.show(5)</code> or <code>display(df.limit(5))</code>	<code>SELECT * FROM t LIMIT 5</code>
Schema	<code>df.printSchema()</code>	<code>DESCRIBE t</code>
Summary statistics	<code>df.describe()</code> , <code>df.summary()</code>	<code>SELECT MIN(x), MAX(x), AVG(x), COUNT(*) FROM t</code>
Row count	<code>df.count()</code>	<code>SELECT COUNT(*) FROM t</code>
Distinct values	<code>df.select("col").distinct()</code>	<code>SELECT DISTINCT col FROM t</code>

SECTION 5

Transforming columns

Selecting columns and expressions

PYSPARK

```
from pyspark.sql.functions import col, upper

df.select("customer_id", "first_name", "email")

df.select(
    col("customer_id"),
    upper(col("first_name")).alias("first_name_upper"),
    col("email"),
)
```

SPARK SQL

```
SELECT customer_id, first_name, email
FROM customers;

SELECT
    customer_id,
    UPPER(first_name) AS first_name_upper,
    email
FROM customers;
```

Adding or changing a column

PYSPARK

```
from pyspark.sql.functions import concat, lit, year

df.withColumn("full_name", concat(col("first_name"), lit(" "), col("last_name")))

df.withColumn("reg_year", year(col("registration_date")))
```

SPARK SQL

```
SELECT *, CONCAT(first_name, ' ', last_name) AS full_name
FROM customers;

SELECT *, YEAR(registration_date) AS reg_year
FROM customers;
```

Changing a type

TRY_CAST returns null instead of raising an error when a value cannot be converted.

PYSPARK

```
df.withColumn("reg_date", col("registration_date").cast(DateType()))

df.withColumn("id_int", col("customer_id").cast("int"))
```

SPARK SQL

```
SELECT CAST(registration_date AS DATE) AS reg_date
FROM customers;

SELECT TRY_CAST(customer_id AS INT) AS id_int
FROM customers;
```

Renaming columns

To rename several columns at once, a single `select` with aliases is clearer than a chain of `withColumnRenamed` calls.

PYSPARK

```
df.withColumnRenamed("customer_id", "id")

df.select(
    col("customer_id").alias("id"),
    col("first_name").alias("fname"),
    col("last_name").alias("lname"),
)
```

SPARK SQL

```
SELECT
    customer_id AS id,
    first_name  AS fname,
    last_name   AS lname
FROM customers;
```

Dropping columns

`SELECT * EXCEPT (...)` is Databricks SQL syntax.

PYSPARK

```
df.drop("email")
df.drop("email", "city", "country")
```

SPARK SQL

```
SELECT * EXCEPT (email, city, country)
FROM customers;
```

SECTION 6

Removing duplicates and sorting

Distinct rows

PYSPARK

```
df.select("country").distinct()

df.select("city", "country").dropDuplicates(["city", "country"])
```

SPARK SQL

```
SELECT DISTINCT country
FROM customers;

SELECT DISTINCT city, country
FROM customers;
```

Sorting

PYSPARK

```
from pyspark.sql.functions import asc, desc

df.orderBy("registration_date")

df.orderBy(asc("country"), desc("registration_date"))
```

SPARK SQL

```
SELECT * FROM customers
ORDER BY registration_date;

SELECT * FROM customers
ORDER BY country ASC, registration_date DESC;
```

SECTION 7

Conditional logic

In PySpark, chain `when()` calls and finish with `otherwise()`. In SQL, the same logic is written as `CASE WHEN ... THEN ... ELSE ... END`, and `COALESCE`, `IFNULL` or `NVL` replace nulls.

PYSPARK

```
from pyspark.sql.functions import when

df.withColumn(
    "customer_tier",
    when(year(col("registration_date")) >= 2023, "New")
    .when(year(col("registration_date")) >= 2021, "Regular")
    .otherwise("Loyal"),
)

df.withColumn(
    "city_safe",
    when(col("city").isNull(), lit("Unknown")).otherwise(col("city")),
)
```

SPARK SQL

```
SELECT *,
    CASE
        WHEN YEAR(registration_date) >= 2023 THEN 'New'
        WHEN YEAR(registration_date) >= 2021 THEN 'Regular'
        ELSE 'Loyal'
    END AS customer_tier
FROM customers;

SELECT *, COALESCE(city, 'Unknown') AS city_safe
FROM customers;
```

SECTION 8

Filtering rows

Simple conditions

`filter()` and `where()` are interchangeable in PySpark.

PYSPARK

```
df.filter(col("country") == "USA")
df.where(col("city") == "New York")

df.filter("country = 'USA'")      # a SQL expression as a string
```

SPARK SQL

```
SELECT * FROM customers WHERE country = 'USA';

SELECT * FROM customers WHERE city = 'New York';
```

Combining conditions

In PySpark, each condition must be in parentheses when conditions are combined with `&` (and) or `|` (or).

PYSPARK

```
df.filter(
    (col("country") == "USA") &
    (year(col("registration_date")) == 2023)
)

df.filter(
    (col("country") == "USA") |
    (col("country") == "UK")
)
```

SPARK SQL

```
SELECT * FROM customers
WHERE country = 'USA'
  AND YEAR(registration_date) = 2023;

SELECT * FROM customers
WHERE country = 'USA'
  OR country = 'UK';
```

Filtering by a list of values

PYSPARK

```
countries = ["USA", "UK", "Germany", "France"]

df.filter(col("country").isin(countries))
df.filter(~col("country").isin(countries))
```

SPARK SQL

```
SELECT * FROM customers
WHERE country IN ('USA', 'UK', 'Germany', 'France');

SELECT * FROM customers
WHERE country NOT IN ('USA', 'UK', 'Germany', 'France');
```

Null values

Comparing with `== None` does not find nulls in PySpark; use `isNull()` or `isNotNull()`.

PYSPARK

```
df.filter(col("email").isNotNull())
df.filter(col("city").isNull())
```

SPARK SQL

```
SELECT * FROM customers WHERE email IS NOT NULL;

SELECT * FROM customers WHERE city IS NULL;
```

Filling or dropping missing values

PYSPARK

```
df.fillna("Unknown", subset=["city"])
df.fillna({"city": "Unknown", "country": "N/A"})

df.dropna()                # rows with any null
df.dropna(how="all")       # rows where every value is null
df.dropna(subset=["customer_id"]) # rows with a null in selected columns
```

SPARK SQL

```
SELECT COALESCE(city, 'Unknown') AS city,
       IFNULL(country, 'N/A')    AS country
FROM customers;

SELECT * FROM customers
WHERE customer_id IS NOT NULL;
```

Matching text

PYSPARK

```
df.filter(col("email").like("%@gmail.com"))
df.filter(col("city").contains("New"))
df.filter(col("first_name").startswith("J"))
df.filter(col("email").endswith(".com"))
df.filter(col("email").rlike(r"^[a-z]+\.[a-z]+"))
```

SPARK SQL

```
SELECT * FROM customers WHERE email LIKE '%@gmail.com';
SELECT * FROM customers WHERE CONTAINS(city, 'New');
SELECT * FROM customers WHERE STARTSWITH(first_name, 'J');
SELECT * FROM customers WHERE ENDSWITH(email, '.com');
SELECT * FROM customers WHERE REGEXP_LIKE(email, '^[a-z]+\.[a-z]+');
```

SECTION 9

Aggregation

Counts, sums and averages

PYSPARK

```
from pyspark.sql.functions import count, sum, avg

df.groupBy("country").count().orderBy(desc("count"))

(orders_df.groupBy("payment_method")
 .agg(sum("total_amount").alias("revenue"),
      avg("total_amount").alias("avg_order"),
      count("*").alias("n_orders"))
 .orderBy(desc("revenue")))
```

SPARK SQL

```
SELECT country, COUNT(*) AS count
FROM customers
GROUP BY country
ORDER BY count DESC;

SELECT
    payment_method,
    SUM(total_amount) AS revenue,
    AVG(total_amount) AS avg_order,
    COUNT(*)          AS n_orders
FROM orders
GROUP BY payment_method
ORDER BY revenue DESC;
```

Minimum and maximum

PYSPARK

```
from pyspark.sql.functions import min, max

orders_df.agg(
    min("total_amount").alias("min_amount"),
    max("total_amount").alias("max_amount"),
    avg("total_amount").alias("avg_amount"),
)
```

SPARK SQL

```
SELECT
    MIN(total_amount) AS min_amount,
    MAX(total_amount) AS max_amount,
    AVG(total_amount) AS avg_amount
FROM orders;
```

Several aggregates in one pass

Computing all aggregates in a single `agg()` call reads the data once.

PYSPARK

```
(orders_df.groupBy("customer_id")
  .agg(count("*").alias("total_orders"),
    sum("total_amount").alias("total_spent"),
    avg("total_amount").alias("avg_order"),
    min("total_amount").alias("min_order"),
    max("total_amount").alias("max_order"))
  .orderBy(desc("total_spent")))
```

SPARK SQL

```
SELECT
  customer_id,
  COUNT(*)          AS total_orders,
  SUM(total_amount) AS total_spent,
  AVG(total_amount) AS avg_order,
  MIN(total_amount) AS min_order,
  MAX(total_amount) AS max_order
FROM orders
GROUP BY customer_id
ORDER BY total_spent DESC;
```

Filtering after aggregation

A `filter()` placed before `groupBy()` corresponds to SQL `WHERE`; a `filter()` placed after `groupBy().agg()` corresponds to `HAVING`.

PYSPARK

```
(orders_df.groupBy("customer_id")
  .agg(count("*").alias("order_count"),
    sum("total_amount").alias("total_spent"))
  .filter(col("order_count") > 5)
  .orderBy(desc("order_count")))
```

SPARK SQL

```
SELECT
  customer_id,
  COUNT(*)          AS order_count,
  SUM(total_amount) AS total_spent
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 5
ORDER BY order_count DESC;
```

SECTION 10

Temporary views and SQL

View type	Visible to	PySpark	Spark SQL	Lifetime
Temporary view	The current session	<code>df.createOrReplaceTempView("v")</code>	<code>CREATE OR REPLACE TEMP VIEW v AS ...</code>	Until the session ends
Global temporary view	All sessions on the cluster	<code>df.createOrReplaceGlobalTempView("v")</code>	Queried as <code>global_temp.v</code>	Until the cluster restarts
View in Unity Catalog	Anyone with privileges	<code>spark.sql("CREATE VIEW ...")</code>	<code>CREATE VIEW catalog.schema.v AS ...</code>	Permanent

Registering and querying a view

PYSPARK

```
customers_df.createOrReplaceTempView("customers")
orders_df.createOrReplaceTempView("orders")

result = spark.sql("""
    SELECT customer_id, COUNT(*) AS order_count
    FROM orders
    GROUP BY customer_id
    ORDER BY order_count DESC
    LIMIT 10
""")
display(result)
```

SPARK SQL

```
CREATE OR REPLACE TEMP VIEW orders AS
SELECT *
FROM read_files('/Volumes/.../orders.json', format => 'json');

SELECT customer_id, COUNT(*) AS order_count
FROM orders
GROUP BY customer_id
ORDER BY order_count DESC
LIMIT 10;
```

Joining in SQL

The same query can be run from Python with `spark.sql()` or directly in a `%sql` cell.

SPARK SQL

```
SELECT
  c.customer_id,
  c.first_name,
  c.country,
  COUNT(o.order_id) AS total_orders,
  SUM(o.total_amount) AS total_spent,
  AVG(o.total_amount) AS avg_order
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = TRY_CAST(o.customer_id AS STRING)
GROUP BY c.customer_id, c.first_name, c.country
HAVING COUNT(o.order_id) > 0
ORDER BY total_spent DESC
LIMIT 10;
```

SECTION 11

Writing to Delta

The PySpark write modes are `overwrite`, `append`, `ignore` and `error`.

PYSARK

```
(df.write
  .format("delta")
  .mode("overwrite")
  .saveAsTable("silver.customers_clean"))

(df.write
  .format("delta")
  .mode("append")
  .saveAsTable("silver.customers_clean"))
```

SPARK SQL

```
CREATE OR REPLACE TABLE silver.customers_clean AS
SELECT * FROM customers
WHERE customer_id IS NOT NULL;

INSERT INTO silver.customers_clean
SELECT * FROM customers_new;
```

SECTION 12

Quick reference

Operation	PySpark	Spark SQL
Read CSV	<code>spark.read.format("csv").option("header", "true").load(path)</code>	<code>read_files(path, format => 'csv', header => true)</code>
Read JSON	<code>spark.read.format("json").load(path)</code>	<code>read_files(path, format => 'json')</code>
Read Parquet	<code>spark.read.format("parquet").load(path)</code>	<code>read_files(path, format => 'parquet')</code>
Infer schema	<code>.option("inferSchema", "true")</code>	Automatic in <code>read_files()</code>
Select columns	<code>df.select("c1", "c2")</code>	<code>SELECT c1, c2 FROM t</code>
New column	<code>df.withColumn("new", expr)</code>	<code>SELECT *, expr AS new FROM t</code>
Rename column	<code>df.withColumnRenamed("old", "new")</code>	<code>SELECT old AS new FROM t</code>
Drop columns	<code>df.drop("c1", "c2")</code>	<code>SELECT * EXCEPT (c1, c2) FROM t</code>
Change type	<code>col("x").cast("int")</code>	<code>CAST(x AS INT), TRY_CAST(x AS INT)</code>
Conditional value	<code>when(cond, v1).otherwise(v2)</code>	<code>CASE WHEN cond THEN v1 ELSE v2 END</code>
Replace null	<code>df.fillna("val", subset=["col"])</code>	<code>COALESCE(col, 'val')</code>
Filter	<code>df.filter(col("x") == "val")</code>	<code>WHERE x = 'val'</code>
And, or	<code>(c1) & (c2), (c1) (c2)</code>	<code>WHERE c1 AND c2, WHERE c1 OR c2</code>
List of values	<code>col("x").isin(["a", "b"])</code>	<code>WHERE x IN ('a', 'b')</code>
Null check	<code>col("x").isNull(), .isNotNull()</code>	<code>WHERE x IS NULL, IS NOT NULL</code>
Pattern	<code>col("x").like("%pat%")</code>	<code>WHERE x LIKE '%pat%'</code>
Regular expression	<code>col("x").rlike(r"^[a-z]+")</code>	<code>WHERE REGEXP_LIKE(x, '^[a-z]+')</code>
Distinct rows	<code>df.distinct(), df.dropDuplicates(["col"])</code>	<code>SELECT DISTINCT ...</code>
Sort	<code>df.orderBy(asc("x"), desc("y"))</code>	<code>ORDER BY x ASC, y DESC</code>
Group	<code>df.groupBy("col").agg(...)</code>	<code>GROUP BY col</code>
Aggregates	<code>count("*"), sum("c"), avg("c"), min("c"), max("c")</code>	<code>COUNT(*), SUM(c), AVG(c), MIN(c), MAX(c)</code>
Filter groups	<code>.filter() after .groupBy().agg()</code>	<code>HAVING COUNT(*) > n</code>
Temporary view	<code>df.createOrReplaceTempView("v")</code>	<code>CREATE OR REPLACE TEMP VIEW v AS ...</code>
SQL from Python	<code>spark.sql("SELECT ...")</code>	<code>%sql cell</code>
Overwrite a table	<code>.write.mode("overwrite").saveAsTable("t")</code>	<code>CREATE OR REPLACE TABLE t AS SELECT ...</code>
Append to a table	<code>.write.mode("append").saveAsTable("t")</code>	<code>INSERT INTO t SELECT ...</code>